

Islamabad AQI Predictor

Technical Report
Islamabad, pakistan

Zaid Afzal

June 8 2026

Abstract

This report documents an end-to-end MLOps system built to predict the Air Quality Index (AQI) for Islamabad. The system pulls hourly weather data from Open-Meteo, stores engineered features in a Hopsworks Cloud Feature Store, and trains an XGBoost regression model that is versioned in the Hopsworks Model Registry. A Streamlit dashboard fetches the latest model and feature data at runtime to deliver live predictions and an interactive EDA suite. The pipeline is designed to run serverlessly — data ingestion and retraining are triggered automatically, requiring zero manual intervention once deployed.

1. Introduction

Islamabad, while often overshadowed by Lahore in air quality discussions, faces among the most polluted cities in the world. DHA Phase 8, a residential area on the city's outskirts, is affected by a mix of vehicle emissions, industrial activity, and seasonal crop-burning smoke drifting in from the Punjab plains. While real-time AQI monitors exist, they tell you what the air is like right now — not what it will be in the next few hours when you might be planning to go outside.

This project addresses that gap by building a forecasting system, not just a monitoring one. The goal is a fully automated pipeline: data comes in, features are computed, a model trains, and the dashboard updates — without anyone having to run a script manually.

1.1 AQI Categories

Level	Category	Health Implication
< 50	Good 	Air quality is satisfactory with little or no risk.
51–100	Moderate 	Acceptable; some pollutants may be a concern for a small number of people.
101–150	Unhealthy 	General public may begin to experience health effects.
151–200	Unhealthy 	Everyone may begin to experience more serious health effects.
> 200	Very Unhealthy 	Health warnings; everyone is likely to be affected.

2. System Architecture

The system follows a straightforward MLOps pattern. Open-Meteo is the sole data source — it provides historical and real-time weather observations (temperature, humidity, wind speed, precipitation, PM2.5 etc.) without requiring an API key. The data lands in a Hopsworks Feature Group, which acts as the feature store. A separate training script reads from that feature group, runs a model tournament, and registers the winning model back to Hopsworks. The Streamlit app then pulls both the model and the latest features at startup.

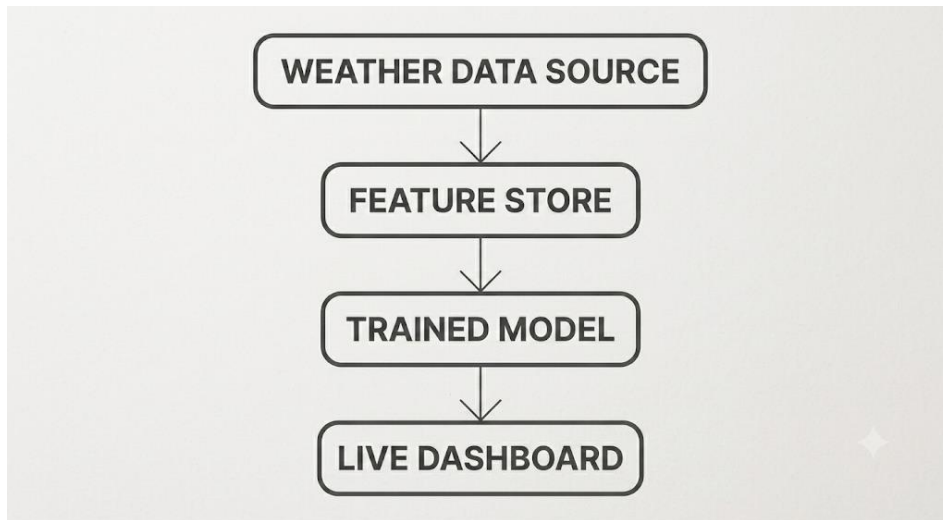


Figure 1 — High-level system architecture

3. Data Pipeline & Feature Engineering

3.1 Scripts

Script	Role
BackFillOpenMeteo.py	One-time historical backfill — fetches months of past weather data and writes to the Hopsworks feature group to give the model enough training rows.
FeaturePipeline.py	Incremental update script — appends new observations to the feature group on a schedule (e.g. hourly via GitHub Actions).
TrainModel.py	Reads all features, runs a champion-selection tournament across Ridge, MLP, and XGBoost, generates SHAP explanations, and registers the best model.

3.2 Feature Set

All features come from Open-Meteo's hourly historical endpoint. The target variable is the AQI value at the current timestamp, derived from the PM2.5 reading using the US EPA breakpoint table.

Category	Features
Pollutants	PM2.5, PM10
Weather	Temperature (°C), Relative Humidity (%), Wind Speed (km/h), Precipitation (mm), Atmospheric Pressure (hPa)
Temporal	Hour of day, Day of week, Month — encoded to capture diurnal and seasonal cycles
Derived	AQI change rate (delta between consecutive readings)

3.3 Target Engineering

The prediction target is the AQI at the current hour. Because the model is meant for next-hour inference, the feature vector presented at prediction time is the most recent row from the feature store, and the output is the expected AQI for the following hour. Rows with null targets are excluded from training.

4. Machine Learning Models

TrainModel.py runs an automated tournament: three models are trained, evaluated on a held-out test set, and the one with the lowest RMSE is crowned champion and registered to Hopsworks. The train/test split is chronological (80/20) to prevent temporal leakage.

4.1 Model Configurations

Model	Key Hyperparameters
Ridge Regression	alpha = 1.0 — baseline linear model.
MLP (Neural Net)	hidden_layer_sizes = (64, 32), max_iter = 500 — captures non-linear interactions.
XGBoost	n_estimators = 100, max_depth = 6, learning_rate = 0.1 — gradient boosted trees, typically wins the tournament.

4.2 Champion Selection

The model with the lowest Root Mean Squared Error on the test set is saved to islamabad_aqi_model_dir/ and uploaded to the Hopsworks Model Registry with RMSE and R² logged as metrics. Because no explicit version is set during registration, Hopsworks auto-increments the version number. The app always fetches the latest version at startup.

5. Model Interpretability (SHAP)

After the champion is selected, `TrainModel.py` generates a SHAP summary plot saved as `shap_summary.png`. SHAP (SHapley Additive exPlanations) attributes each feature a contribution score for every prediction, giving a transparent view of what the model is actually relying on.

For XGBoost (the usual winner), a `TreeExplainer` is used, which is exact and fast. For Ridge or MLP, a `KernelExplainer` with 100 background samples is used instead. The summary plot is displayed in the Explainable AI tab of the Streamlit dashboard.

Based on preliminary runs, the features with the highest mean |SHAP| values are typically PM2.5 concentration, hour of day (rush-hour effect), and month (seasonal crop-burning cycles). Wind speed shows a negative SHAP direction, consistent with its role in dispersing pollutants.

6. Streamlit Dashboard

The `app.py` dashboard is the user-facing component. On startup it calls two cached functions — `load_model()` and `load_history()` — both of which connect to Hopsworks. The model is downloaded locally and loaded with `joblib`; the feature history is read directly into a `DataFrame`.

6.1 Dashboard Sections

Section	What It Shows
Live Cloud Telemetry	Four metric cards: current AQI, PM2.5, temperature, humidity — all from the most recent feature store row.
Predictive ML Inference	A button that runs <code>model.predict()</code> on the latest feature vector and displays the predicted next-hour AQI with a colour-coded category label.
Traffic & Time Analysis	Bar chart of average AQI by hour, showing morning and evening rush-hour peaks.
Weather Correlation Analytics	Line chart for any selected feature over time, plus a correlation coefficient table against AQI.
Explainable AI (SHAP)	Displays the <code>shap_summary.png</code> generated during training, or a warning if the file is absent.

7. Deployment & Automation

The intended deployment model is fully serverless. The pipeline scripts are triggered by GitHub Actions on a cron schedule — `FeaturePipeline.py` runs hourly, `TrainModel.py` runs daily. The Streamlit app is hosted on Streamlit Community Cloud, where the `HOPSWORKS_API_KEY` is stored as a secret and read via `st.secrets` at runtime.

7.1 GitHub Actions Secrets Required

Secret	Purpose
HOPSWORKS_API_KEY	Authenticates all Hopsworks SDK calls (feature store read/write, model registry).

7.2 Running Order (First Time)

- 1. BackFillOpenMeteo.py — populates the feature group with historical data.
- 2. TrainModel.py — trains models, registers champion to Hopsworks.
- 3. app.py (Streamlit) — can now find the model and feature data at startup.
- 4. Schedule FeaturePipeline.py (hourly) and TrainModel.py (daily) via GitHub Actions.

8. Challenges Encountered

8.1 Model Version Mismatch

The most time-consuming bug was a mismatch between the version registered by TrainModel.py and the version requested in app.py. TrainModel.py registers the model without an explicit version, so Hopsworks assigns one automatically (starting at 1). The app was hardcoded to request version 3, which didn't exist — causing get_model() to return None and the subsequent .download() call to crash with 'NoneType object has no attribute download'. The fix was simple: remove the version argument from get_model() so the app always fetches the latest available version regardless of how many times the model has been retrained.

8.2 Hopsworks Free Tier

Hopsworks previously offered a free sandbox tier. During development this was removed in favour of a pay-as-you-go model, which required re-evaluating the cloud storage approach. After some back-and-forth with support, access was restored, but it added a delay to the project timeline.

8.3 Open-Meteo Data Gaps

Open-Meteo's historical endpoint occasionally returns incomplete hourly rows for the most recent 48-hour window. These rows were dropped during preprocessing. For the backfill, data was fetched in monthly chunks to stay within the API's rate limits without requiring a paid plan.

9. Conclusion

The system works end-to-end: weather data flows in automatically, a model is trained and versioned, and a dashboard serves live predictions. The main lesson from the development process is that version management between the training pipeline and the serving layer needs to be explicit from the start — hardcoding a version number that doesn't yet exist is the kind of bug that's obvious in hindsight but hard to spot when you're focused on the model training side.